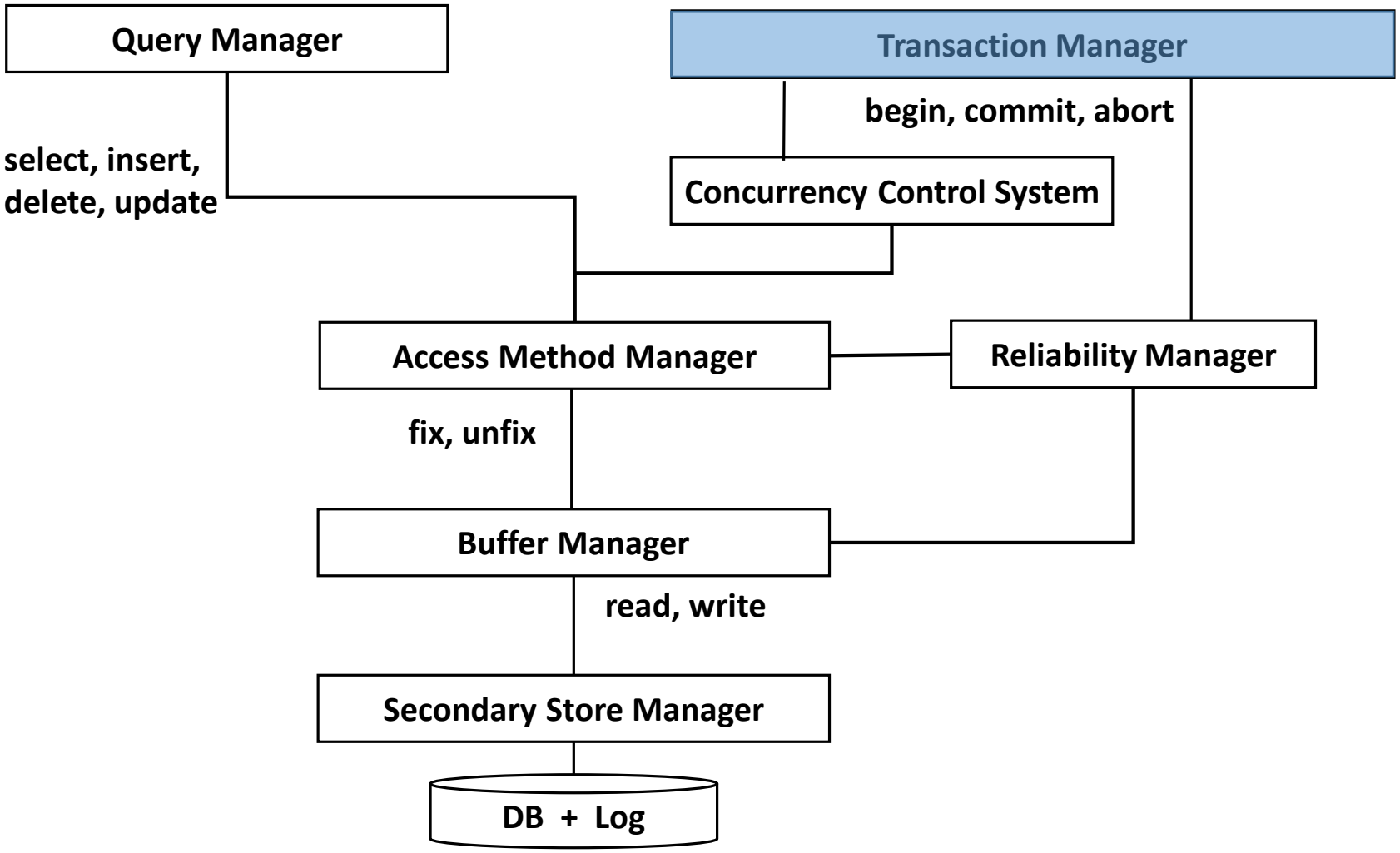
Data bases 2

Transactional Systems

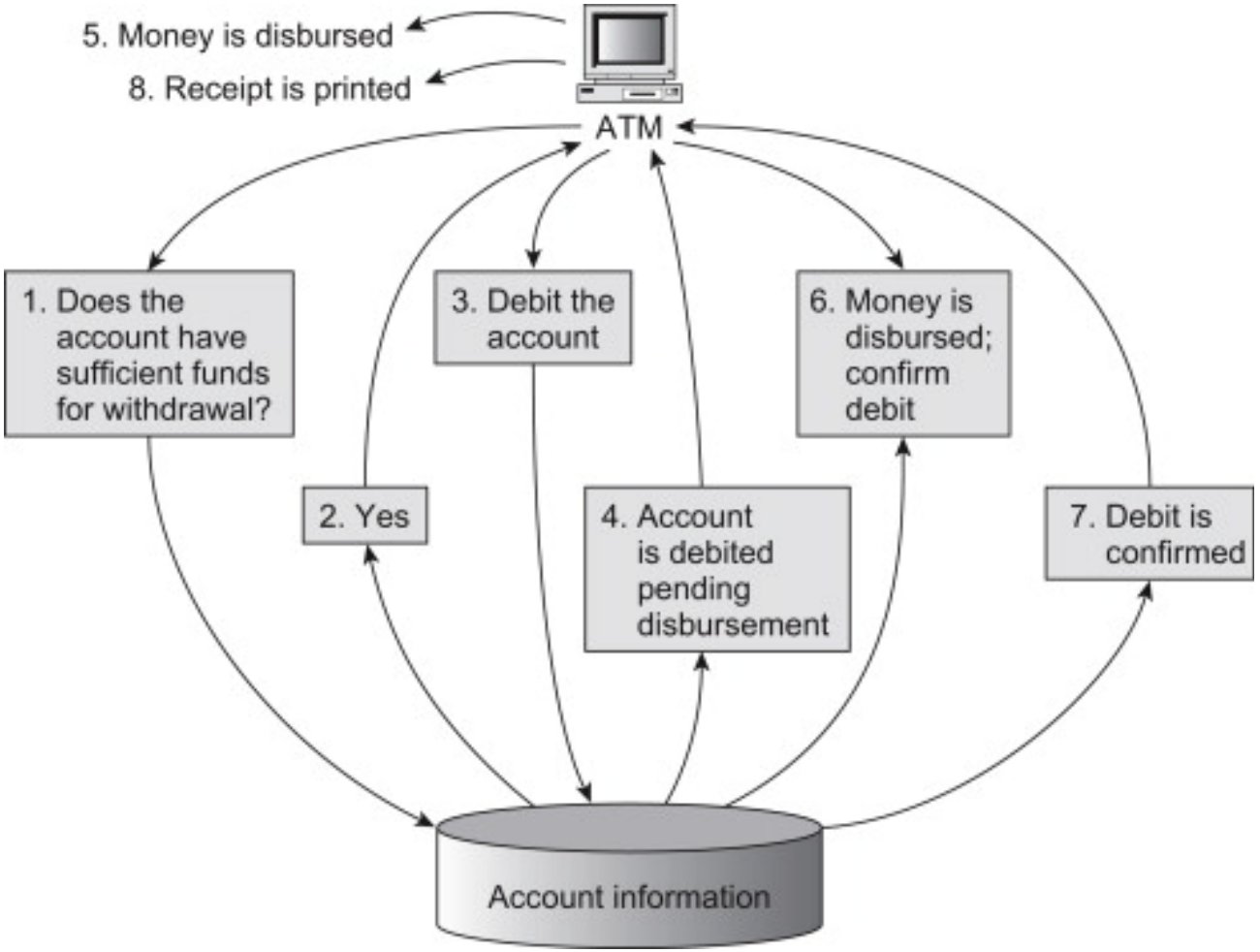
In this lecture

- The architecture of the DDBMS
- Introduction to the concept of transaction
- Properties of a transaction
- Modules of the DBMS responsible of guaranteeing such properties

The architecture of the DBMS



The need of transactions: ATM example

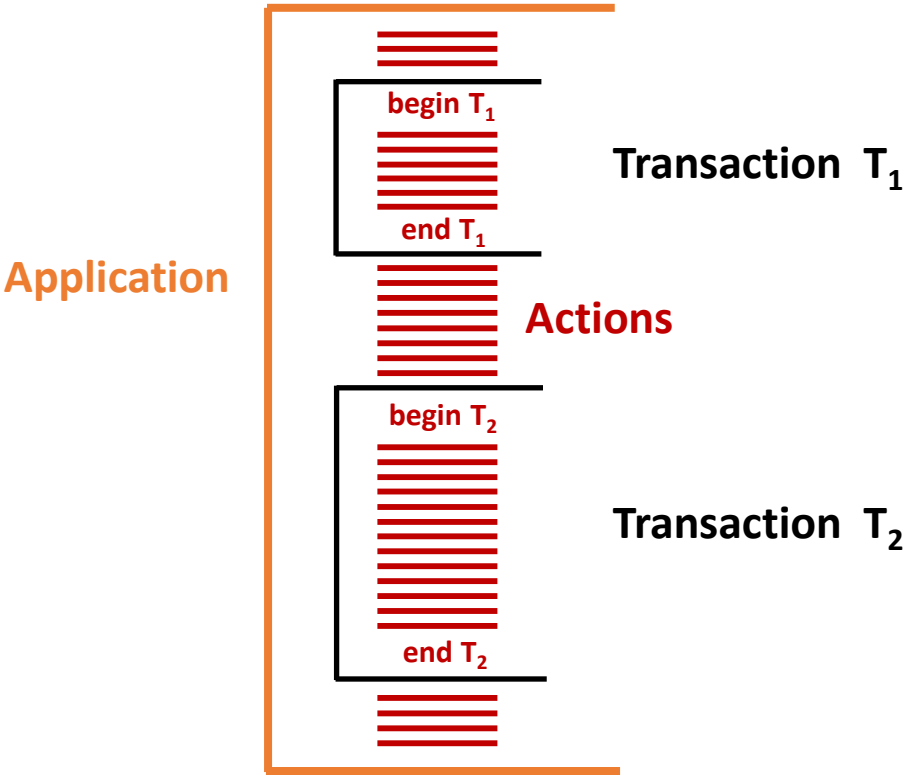


Definition of Transaction

- An elementary, atomic unit of work performed by an application
- Each transaction is conceptually encapsulated within two commands:
  - begin transaction
  - commit-work (commit) or rollback-work (rollback)

    Standard SQL Commonly used
- Transactional System (OLTP): a system that supports the execution of transactions on behalf of concurrent applications

Difference between Application and Transaction



Bank transfer

```
BEGIN TRANSACTION;

UPDATE Account
SET Balance = Balance - 10
WHERE AccNum = 42177;

UPDATE Account
SET Balance = Balance + 10
WHERE AccNum = 12202;

COMMIT;
```

Event registration with limited seats

```
BEGIN TRANSACTION;
DECLARE P INTEGER;
SELECT available_seats INTO P
   FROM Event
   WHERE event_id = 12
   FOR UPDATE;
IF P > 0 THEN
    UPDATE Event
      SET available_seats = available_seats - 1
      WHERE event_id = 12;
    INSERT INTO Reservation(reservation_id, event_id, customer_id)
      VALUES (501, 12, 34);
    COMMIT;
ELSE
    ROLLBACK;
END IF;
```

FOR UPDATE temporarily *locks* the event row until the end of the transaction, preventing another user from performing the same operation while a user is checking and decreasing the number of available seats.

Remark: this example uses procedural pseudocode (variable declaration, IF). In practice, this logic may be implemented in application code or in a stored procedure

ACID Properties of Transactions

- A transaction is a unit of work characterized by the following properties:

  - **A**tomicity
  - **C**onsistency
  - **I**solation
  - **D**urability

Atomicity

- A transaction is an indivisible unit of execution
  - Either all the operations in the transaction are executed or none is executed
- In the case of the bank transfer, the execution of a single update statement would be disastrous
- The time in which COMMIT is executed marks the instant in which the transaction ends successfully:
  - An error before should cause the rollback of the work
  - An error afterwards should not alter the effect of the transaction
- The ROLLBACK of the work performed can be caused
  - By the application with a ROLLBACK statement
  - By the DBMS, for example for the violation of integrity constraints or for concurrency management
- In case of a rollback, the work performed must be undone, bringing the database to the state it had before the start of the transaction
- It is the application's responsibility to decide whether an aborted transaction must be redone or not

Consistency

- A transaction must preserve the database integrity constraints.

- Integrity constraints are normally checked at the end of each SQL statement. In such a case, each intermediate state must satisfy them.

- When constraints are declared DEFERRABLE, their verification can be postponed until COMMIT. In such a case, if the initial state S_0 is consistent, then the final state S_f must also be consistent.
  - The types of constraints that can be deferred depend on the DBMS
    - In PostgreSQL, UNIQUE, PRIMARY KEY, FOREIGN KEY and EXCLUDE can be deferred. CHECK and NOT NULL are checked at the end of each statement.
    - Integrity constraints are always checked immediately by MySQL.

Unique desk number

id	name	desk
1	Alice	10
2	Bob	20

```
CREATE TABLE Employee (
        employee_id INTEGER PRIMARY KEY,
        name VARCHAR(50) NOT NULL,
        desk_number INTEGER NOT NULL,
        CONSTRAINT unique_desk_number  UNIQUE (desk_number)
        DEFERRABLE INITIALLY DEFERRED
);
```

```
BEGIN TRANSACTION;

UPDATE Employee
SET desk_number = 20
WHERE employee_id = 1;

UPDATE Employee
SET desk_number = 10
WHERE employee_id = 2;

COMMIT;
```

-- Temporary state: desk 20 is assigned to both employees

-- Final state: each desk is assigned to one employee

Allocated hours constraint

A project has 40 planned hours.

Task 1: 20 allocated hours
Task 2: 20 allocated hours

```
CREATE CONSTRAINT TRIGGER project_hours_constraint
AFTER INSERT OR UPDATE OF allocated_hours ON Task
DEFERRABLE INITIALLY DEFERRED
FOR EACH ROW
…
```

Constraint: sum of the allocated hours of all tasks = planned hours of the project

```
BEGIN TRANSACTION;

UPDATE Task
SET allocated_hours = allocated_hours - 5
WHERE task_id = 1;

UPDATE Task
SET allocated_hours = allocated_hours + 5
WHERE task_id = 2;

COMMIT;
```

-- Temporary state: 15 + 20 = 35
constraint violated

-- Final state: 15 + 25 = 40
constraint satisfied

Isolation

- The execution of a transaction must be independent of the concurrent execution of other transactions
- In particular, the concurrent execution of a number of transactions **must produce the same result as the execution of the same transactions in a sequence**
- E.g., the concurrent execution of transactions T1 and T2 must produce the same results that can be obtained by executing one of these sequences
  - T1,T2
  - T2,T1
- Isolation impacts performance and trade-offs can be defined between isolation and performance

Durability

- The effect of a transaction that has successfully committed will last "forever"
  - Independently of any system fault
  - Sounds obvious… but every DBMS manipulates data in main memory

ACID properties & DBMS mechanisms

- Atomicity
  - Abort-rollback-restart, commit protocols
  - Reliability Manager
- Consistency
  - Integrity checking of the DBMS
  - Integrity Control System at query execution time (with the support of the DDL compiler)
- Isolation
  - Concurrency control
  - Concurrency Control System
- Durability
  - Recovery management
  - Reliability Manager

Transactions & DBMS architecture

